



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez.

Asignatura: Programación orientada a objetos.

Grupo: 07

No de Práctica(s): 3

Integrante(s): 323331229

426060019

323217156

312109239

323195386

321321976

*No. de lista o
brigada:* 6

Semestre: 3^{ro}

Fecha de entrega: 11/09/2026

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	3
1.1. Planteamiento del problema	3
1.2. Motivación	3
1.3. Objetivos	3
2. Marco Teórico	4
3. Desarrollo	5
3.1. Ejercicio 1	5
3.2. Ejercicio 2	6
4. Resultados	8
4.1. Resultados del Ejercicio 1	8
4.2. Resultados del Ejercicio 2	9
5. Conclusiones	11

1. Introducción

1.1. Planteamiento del problema

Esta práctica se conformó de dos retos. El primer problema es investigar preguntándole a nuestro LLM el funcionamiento de un HashMap y un TreeSet además pedirle 3 ejemplos comunes de aplicación. El segundo problema consiste en diseñar una aplicación en java que lea un numero entero de 5 dígitos y determine si es un palíndromo, utilizando los siguientes parámetros: Si el número no tiene 5 dígitos mostrar mensaje de error y permitir al usuario ingresar un nuevo valor (no valen negativos) y requiriendo al menos un método estático.

1.2. Motivación

Resolver estos problemas es importante para construir habilidades ya que, al saber seleccionar la estructura de datos adecuada, como un mapa o un árbol logramos que una aplicación funcione bien y sea eficiente en entornos profesionales. Además, aprender a trabajar con herramientas de Inteligencia Artificial, como los LLM, es hoy en día una competencia esencial para avanzar más rápido en proyectos. Por otro lado, desarrollar el programa en Java ayuda a promover buenas prácticas de programación, como validar cuidadosamente las entradas del usuario para prevenir errores graves en el sistema, y el uso de métodos estáticos, para escribir código claro y que se pueda reutilizar fácilmente.

1.3. Objetivos

- **General:** Aprender a usar la Inteligencia Artificial para entender mejor cómo funcionan las herramientas de programación, como HashMap y TreeSet. Probar nuestra lógica creando un programa en Java que encuentre números palíndromos sin permitir que el usuario ingrese datos incorrectos y utilizando métodos estáticos.
- **Específicos:**
 - Analizar el funcionamiento interno de las colecciones HashMap y TreeSet, apoyándose en un LLM.
 - Identificar al menos tres escenarios prácticos y reales de aplicación para cada una de estas estructuras de datos.
 - Diseñar e implementar un algoritmo en Java capaz de calcular si una numero de 5 digitos ingresado por el usuario es un palíndromo.
 - Desarrollar un sistema de validación de entradas que restrinja el programa exclusivamente a números enteros positivos de cinco dígitos, solicitando el reingreso de datos en caso de no cumplir.
 - Aplicar la creación y ejecución de al menos un método estático.

2. Marco Teórico

1. Palíndromo: Palabra o frase cuyas letras están dispuestas de tal manera que resulta la misma leída de izquierda a derecha que de derecha a izquierda. [1]
2. Arreglos: Es un objeto que permite guardar varios datos del mismo tipo. Cada dato en el arreglo tiene una posición numerada, y la primera posición es el 0. [3]
3. Wrappers: Los wrappers o clases envoltorio son clases diseñadas para ser un complemento de los tipos primitivos, los tipos primitivos son los únicos elementos de Java que no son objetos. Existe una clase wrapper para cada uno de los tipos primitivos: Byte, Short, Character, Integer, Long, Float, Double y Boolean. Todas las clases wrapper permiten crear un objeto de la clase a partir de tipo básico. [2]
4. Colecciones: Es un objeto que guarda un grupo de referencias a otros objetos, como si fuera un arreglo con objetos, pero a diferencia de los arreglos, las colecciones cambian de tamaño según sea necesario. Permiten agregar y quitar objetos mientras se ejecuta el programa. [2]
5. Conjunto: Un conjunto (Set) es una colección desordenada (no mantiene un orden de inserción) y no permite elementos duplicados. [4]
6. Mapas: Es una colección de elementos organizados en pares, donde cada par tiene una llave y un valor. Las llaves no se pueden repetir, cada valor está asociado a una llave única. Los valores sí pueden aparecer más de una vez. [4]

3. Desarrollo

3.1. Ejercicio 1

Para este ejercicio le preguntamos a nuestro LLM el funcionamiento de un HashMap y un TreeSet junto con 3 ejemplos más comunes de aplicación.

Respecto al HashMap, el LLM explicó que es una estructura que almacena datos en pares de clave-valor y que su objetivo principal es encontrar un valor específico casi al instante, sin importar cuántos millones de registros haya guardados.

Su funcionamiento es:

1. Cuando insertas una clave, el HashMap pasa esa clave por una función hash matemática.
2. Esa función convierte la clave (ej. "Juan") en un número entero (un índice).
3. Ese índice determina en qué cubeta "(bucket o espacio de memoria) se guardará el valor.
4. Cuando quieres buscar a "Juan", se repite la función matemática, y el sistema sabe exactamente a qué cubeta ir a buscar. No hay orden lógico entre los elementos; están distribuidos donde el algoritmo matemático decidió.

Como ejemplos más comunes de aplicación menciona los siguientes tres:

- Sistemas de caché: Guardar resultados de operaciones costosas (ej. consultas a base de datos). La clave es la consulta, el valor es el resultado. Si la misma consulta llega, se devuelve el valor almacenado instantáneamente.
- Conteo de frecuencias: Ideal para saber cuántas veces se repite algo. Por ejemplo, al analizar un texto, la clave es la palabra (el) y el valor es cuántas veces aparece (45 veces).
- Tablas de búsqueda o diccionarios: Cualquier sistema que requiera buscar atributos por un identificador único. Por ejemplo, buscar los detalles de un usuario en un servidor usando su ID y Usuario como clave.

Respecto al TreeSet, el LLM nos explica que es una estructura que pertenece a la familia de los Sets (conjuntos). Esto significa que no permite elementos duplicados y en el caso específico del TreeSet, siempre mantiene sus elementos ordenados. Su funcionamiento es:

1. Cuando insertas un número (ej. 31), el árbol compara el número con el "nodo raíz"(ej. 70).
2. Si es menor, lo envía a la rama izquierda; si es mayor, a la derecha.
3. El árbol se reacomoda (se autobalancea) constantemente para asegurarse de que no quede asimétrico. Al recorrer el árbol de izquierda a derecha, los datos salen perfectamente ordenados.

En ejemplos de aplicación menciono los siguientes:

- Leaderboards (Tablas de clasificación): Si estás desarrollando un juego y necesitas mantener una lista de las puntuaciones más altas (sin que la misma puntuación de un usuario se repita), el TreeSet insertará la nueva puntuación y la ordenará automáticamente de mayor a menor.
- Filtros de datos únicos y ordenados: Si recibes un millón de correos electrónicos de una base de datos donde muchos están repetidos y desordenados, al pasarlos todos por un TreeSet, los duplicados serán ignorados y el resultado final será una lista limpia y ordenada alfabéticamente.
- Agendas o sistemas de reservas: Para guardar fechas u horarios de eventos. Al estar siempre ordenados temporalmente, es muy rápido y eficiente preguntarle al árbol: "¿Cuál es el evento más cercano después de esta fecha en particular?".

3.2. Ejercicio 2

El ejercicio 2 consiste en diseñar una aplicación en java que lea un numero entero de 5 dígitos y determine si es un palíndromo. Respetando 2 requisitos: Si el número no tiene 5 dígitos mostrar mensaje de error y permitir al usuario ingresar un nuevo valor (no valen negativos). Se requieren métodos estáticos al menos uno. Sabiendo esto utilizamos la lógica vista en clase como el camino fácil sin usar wrappers. La cual es la siguiente:

1. El algoritmo comienza verificando que el número ingresado tenga exactamente 5 dígitos.
2. Hasta que el número sea mayor a 9999 y menor a 100000. Si ingresas un número de 4 dígitos, por ejemplo 9,999 o un número de 6 dígitos, como 100,000 el algoritmo no avanzará y volverá a pedir el dato.
3. Se define el método evaluarPalindromo como estático. Esto quiere decir que puede usarse sin necesidad de crear un objeto primero. El método devuelve un valor booleano, es decir, solo puede devolver true o false.
4. En lugar de buscar en posiciones de memoria usamos aritmética para recortar los pedazos del número que nos interesaban.
5. Primer dígito: Al hacer una división entera entre 10,000 se eliminan los últimos cuatro dígitos.
6. Segundo dígito: Primero divide entre 1,000 para quedarse con los dos primeros dígitos luego le aplica módulo 10 a ese resultado para atrapar el último número.
7. Cuarto dígito: Divide entre 10 para borrar el último dígito después, saca el módulo 10 para quedarse con el nuevo último dígito que era el cuarto originalmente.
8. Quinto dígito: El módulo 10 siempre devuelve el residuo de la división, el residuo siempre es el último número.
9. El algoritmo ignora el tercer dígito ya que al ser el centro de un número impar de cifras siempre será igual a sí mismo.

10. Comprueba que los extremos coincidan ($d1=d5$) Y que los números internos también coincidan ($d2=d4$). Ambas condiciones tienen que ser verdaderas
11. Si ambas condiciones se cumplen la función devuelve True; de lo contrario devuelve False.
12. Finalmente el programa muestra en pantalla si el número ingresado es o no un palíndromo.

Como prueba, validamos el programa con dos palíndromos de ejemplo (12321, 55555), confirmando que el programa los identificara correctamente como palíndromos; se probó con un número de cinco dígitos que no es palíndromo (12345), verificando que el programa lo reportara correctamente como tal. También se probaron los casos de validación: un número con menos de cinco (1906) dígitos y un número negativo (-5), comprobando en ambos casos que el programa mostrara el mensaje de error y permitiera reingresar el valor.

4. Resultados

4.1. Resultados del Ejercicio 1

Respuesta obtenida al consultar con nuestro LLM sobre el funcionamiento de HashMap, TreeSet y los ejemplos de aplicación solicitados.

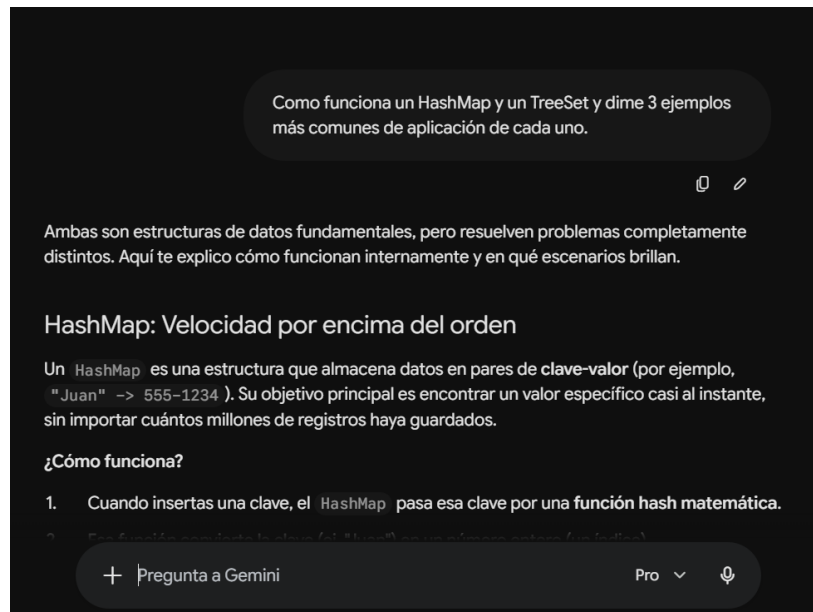


Figura 1: Respuesta del LLM.

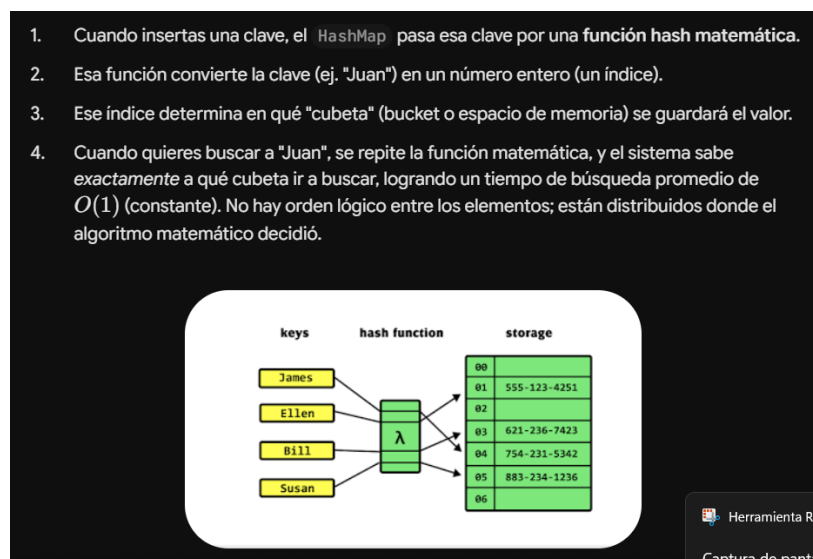


Figura 2: Respuesta del LLM sobre HashMap.

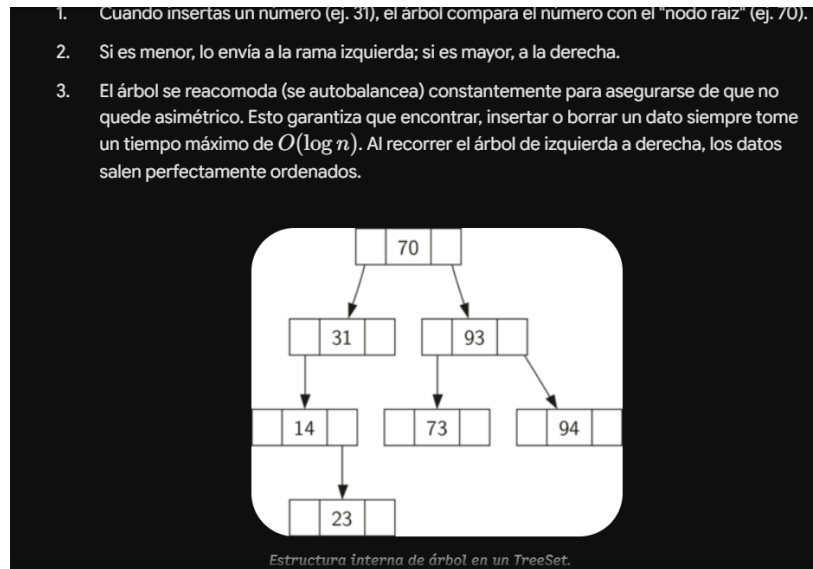


Figura 3: Respuesta del LLM sobre TreeSet.

4.2. Resultados del Ejercicio 2

Ejecución del programa con los números palíndromos de ejemplo en el enunciado: (12321, 55555).

```
PS C:\Users\o-ale\P00\P00> java Ejercicio2
Ingresa un numero entero de cinco digitos: 12321
El numero 12321 es un palindromo.
PS C:\Users\o-ale\P00\P00> |
```

Figura 4: Verificación de números palíndromos (12321).

```
PS C:\Users\o-ale\P00\P00> java Ejercicio2
Ingresa un numero entero de cinco digitos: 55555
El numero 55555 es un palindromo.
PS C:\Users\o-ale\P00\P00> |
```

Figura 5: Verificación de números palíndromos (55555).

Ejecución del programa con un número que no es palíndromo.

```
PS C:\Users\o-ale\P00\P00> java Ejercicio2
Ingresa un numero entero de cinco digitos: 12345
El numero 12345 no es un palindromo.
PS C:\Users\o-ale\P00\P00> |
```

Figura 6: Número que no es palíndromo.

Validación del programa ante entradas incorrectas: un número con menos de cinco dígitos (1906) y un número negativo (-5).

```
Ingresa un numero entero de cinco digitos: 1906
Error: El numero no tiene cinco digitos. Intenta de nuevo.
Ingresa un numero entero de cinco digitos: -5
Error: El numero no tiene cinco digitos. Intenta de nuevo.
Ingresa un numero entero de cinco digitos: |
```

Figura 7: Validación de entradas incorrectas.

5. Conclusiones

El análisis de los resultados que obtuvimos consultando al LLM ayudó a comprender mejor el funcionamiento interno de las colecciones en Java, al hacer dicha consulta preguntando a distintas inteligencias artificiales obtuvimos respuestas distintas pero manteniendo la idea básica de las colecciones consultadas, en un breve resumen, el conocimiento que se obtuvo fue: el uso de un HashMap resulta fundamental cuando se prioriza una velocidad de búsqueda casi instantánea mediante el uso claves asociadas a valores, mientras que un TreeSet es la opción ideal para mantener un conjunto de elementos únicos y ordenarlos de manera automática.

La resolución del programa para evaluar números palíndromos demostró que es posible desarrollar algoritmos eficientes utilizando aritmética básica en lugar de depender de clases wrappers o “envolvedores”, en este caso, el uso de divisiones enteras y el operador módulo 10 (%10) garantizó la “extracción” precisa de los dígitos necesarios para comparar los extremos y las posiciones internas del número ingresado, esto puede resultar útil para la optimización del código porque hace los cálculos más rápidamente y ya no hace falta hacer llamadas externas a utilerías en otras bibliotecas.

Referencias

- [1] Palíndromo. (s. f.). Dle. Recuperado 10 de septiembre de 2026, de <https://dle.rae.es/pal>
- [2] Programador Certificado Java 2. (2.a ed.). (1992). Alfaomega Grupo Editor.
- [3] Introducción a la programación con Java (1.a ed.). (2009). Mc Graw Hill.
- [4] SCJP Sun Certified Programmer for Java 6 Study Guide. (8a. C.). Mc Graw Hill.